pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Case Study: Treasury Yields

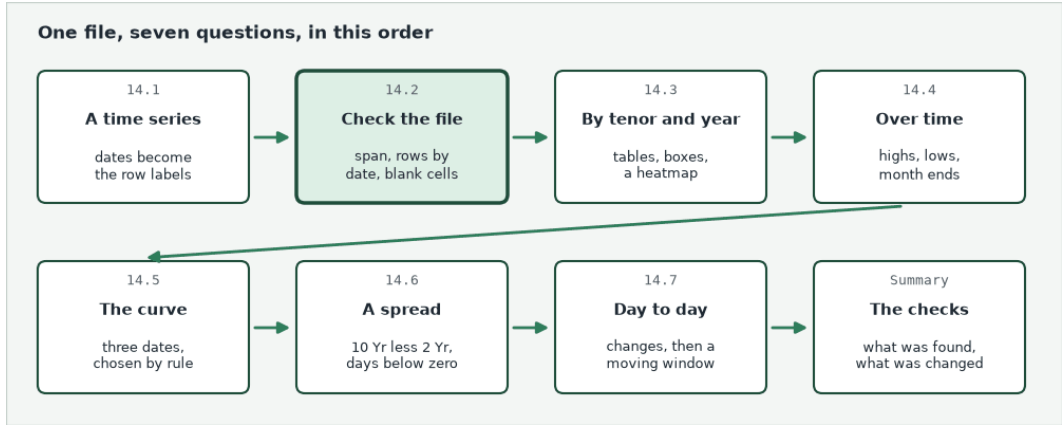
WEEK 3, DAY 4

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's note, and Treasury's own file

Our investment grade bonds are quoted over Treasuries, and the desk's Treasury sheet is stale. Rebuild it from Treasury's own file, so it reruns on every download. Tell me what the file covers and what is missing, where each tenor has been by year, what the curve looked like, the 10s less 2s spread and how long it stayed below zero, and how much yields move in a day. Describe it: no why, no forecast.



The course's first real file: the portfolio is invented, these yields are Treasury's.

Source	U.S. Department of the Treasury, Daily Treasury Par Yield Curve Rates
Retrieved	2026-10-03
Span	2015-01-02 to 2026-10-02
File	treasury_par_yields.csv in data, built by tools/get_treasury_yields.py

As published: nothing filled or removed; only the layout changed, dates oldest first. Treasury builds these par yields from closing bid prices on the most recently auctioned securities. Columns: "date", then fourteen tenors from "1 Mo" to "30 Yr"; "1.5 Month" is Treasury's own label.

Yields in percent; spreads and daily changes in basis points, the difference in percent times 100.

Seven questions, one file, one rule: every sentence is a number the file supports.

14.1 Dates become the row labels

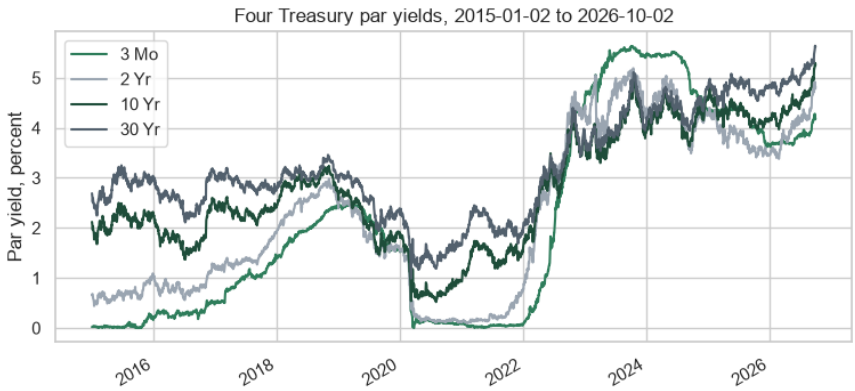
```
raw = pd.read_csv(data_folder + 'treasury_par_yields.csv')
print(raw.shape)
raw.head(3)
(2940, 15)
```

Three tenors start blank, "NaN"; "date" arrives as text, since a CSV holds no types, as Combining Part 2 showed.

```
raw.index.year
AttributeError: 'RangeIndex' object has no attribute 'year'
```

```
yields = pd.read_csv(data_folder + 'treasury_par_yields.csv', parse_dates=['date'],
                     index_col='date')
tenors = list(yields.columns)
print(yields.shape, type(yields.index))
print(yields.index.is_monotonic_increasing, yields.index.is_unique)
print(yields.index[0].date(), 'to', yields.index[-1].date())
(2940, 14) <class 'pandas.DatetimeIndex'>
True True
2015-01-02 to 2026-10-02
```

"date" became a "DatetimeIndex", an index of dates: a table with dates as row labels is a time series. In order and unique, so each row's change from the row above runs forward in time.



The "3 Mo" sits at the bottom for most of 2015 to 2021, then on top through 2023 and most of 2024.

Excel side by side

Excel 365 converts the dates on opening, so "=ISNUMBER(A2)" is TRUE; column I holds the "2 Yr", M the "10 Yr".

A date index: pick by date, group by year, draw against time. Now check the file.

14.2 One date, then the weekdays with no row

```
yields.loc['2020-03-09', main]
3 Mo     0.33
2 Yr     0.38
10 Yr    0.54
30 Yr    0.99
Name: 2020-03-09 00:00:00, dtype: float64
```

One row is one date's curve.

```
print(yields.groupby(yields.index.dayofweek).size().to_dict())
{0: 538, 1: 608, 2: 604, 3: 596, 4: 594}
```

Only weekdays appear, 538 Mondays against 608 Tuesdays: several holidays fall on a Monday.

```
print('2024-07-04' in yields.index, '2024-07-05' in yields.index)
weekdays = pd.date_range(yields.index[0], yields.index[-1], freq='B')
no_row = weekdays.difference(yields.index)
print(len(weekdays), len(yields), len(no_row))
False True
3066 2940 126
```

Two kinds of missing

Absent: a weekday with no row

date	3 Mo	2 Yr	10 Yr
2024-07-03	5.47	4.71	4.36
2024-07-04: no row at all			
2024-07-05	5.46	4.60	4.28

No cell says it is missing.
Found only against a calendar: pd.date_range(freq='B'), difference().

Blank: a row with a cell empty

date	1 Mo	2 Mo	3 Mo
2018-10-12	2.14	NaN	2.28
2018-10-15	2.17	NaN	2.31
2018-10-16	2.19	2.22	2.30

The row is there; the cell holds NaN.
Found by isna(); the first value by first_valid_index().

The span has 3,066 weekdays and the file 2,940 rows: 126 weekdays are absent, twelve of them in 2024.

Excel side by side.

Job	pandas	Excel
One date	`yields.loc['2020-03-09', '10 Yr']`	=XLOOKUP(DATE(2020,3,9), A2:A2941, M2:M2941), Excel 2021 on
Weekdays	`pd.date_range(freq='B')`	=NETWORKDAYS(A2, A2941), 3,066
Blanks	`isna().sum()`	=COUNTBLANK(D2:D2941), 949

Blank cells: three tenors that start late

```
blanks = yields.isna().sum()
print('Blank cells:', blanks.sum())
for tenor in ['2 Mo', '4 Mo', '1.5 Month']:
    first = yields[tenor].first_valid_index()
    print(tenor, blanks[tenor], 'blank | first value', first.date(),
          '| blank after it:', yields.loc[first:, tenor].isna().sum())
Blank cells: 5432
2 Mo 949 blank | first value 2018-10-16 | blank after it: 0
4 Mo 1951 blank | first value 2022-10-19 | blank after it: 0
1.5 Month 2532 blank | first value 2025-02-18 | blank after it: 0
```

```
year_2018 = yields.loc['2018']
same_days = year_2018['2 Mo'].notna()
print('2 Mo:', year_2018['2 Mo'].count(), 'values, mean', round(year_2018['2 Mo'].mean(), 2))
print('3 Mo, whole year:', round(year_2018['3 Mo'].mean(), 2))
print('3 Mo, same days:', round(year_2018.loc[same_days, '3 Mo'].mean(), 2))
2 Mo: 51 values, mean 2.33
3 Mo, whole year: 1.97
3 Mo, same days: 2.38
```

Why three columns start late

Treasury's bill timeline lists the first auction of 8-week bills in October 2018, of 17-week bills in October 2022, and of 6-week bills in February 2025. The "2 Mo", "4 Mo", and "1.5 Month" first have values in those months. Treasury builds the curve from the most recently auctioned bills, so a tenor has no par yield before its bill exists.

Sources: TreasuryDirect, bills timeline; Treasury, yield curve methodology

All 5,432 blanks fall before a late tenor's first date: nothing is scattered, so nothing needs filling.

Its 51 values are all from late 2018: on those days the "3 Mo" averaged more, 2.38.

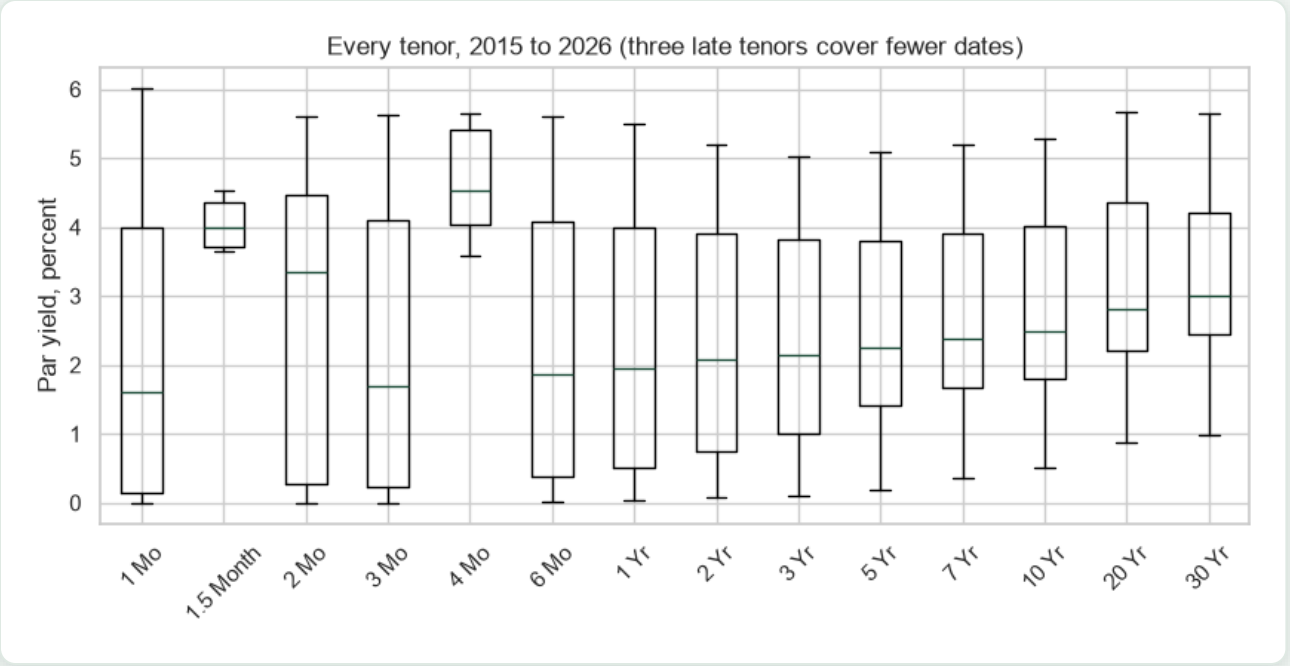
126 days absent, 5,432 cells blank, nothing filled: put "count()" beside every average.

14.3 Every tenor at once

```
yields.describe().T[['count', 'mean', 'std', 'min', '50%', 'max']].round(2)
```

	count	mean	std	min	50%	max
1 Mo	2940.0	2.10	1.98	0.00	1.62	6.02
1.5 Month	408.0	4.02	0.29	3.65	4.00	4.53
2 Mo	1991.0	2.82	2.02	0.00	3.34	5.61
3 Mo	2940.0	2.16	1.96	0.00	1.69	5.63
4 Mo	989.0	4.66	0.66	3.58	4.52	5.64
6 Mo	2940.0	2.22	1.91	0.02	1.87	5.61
1 Yr	2940.0	2.24	1.79	0.04	1.94	5.49
2 Yr	2940.0	2.28	1.60	0.09	2.08	5.19
3 Yr	2940.0	2.34	1.48	0.10	2.15	5.03
5 Yr	2940.0	2.47	1.32	0.19	2.25	5.09
7 Yr	2940.0	2.64	1.24	0.36	2.38	5.19
10 Yr	2940.0	2.76	1.19	0.52	2.49	5.29
20 Yr	2940.0	3.13	1.17	0.87	2.80	5.68
30 Yr	2940.0	3.22	1.06	0.99	3.00	5.64

The "count" column shows the three late tenors. The "10 Yr" ran 0.52 to 5.29, mean 2.76 above median 2.49.



Among the eleven full tenors, medians step up from 1.62 at the "1 Mo" to 3.00 at the "30 Yr". The three late boxes sit high: their dates start in 2018, 2022, and 2025.

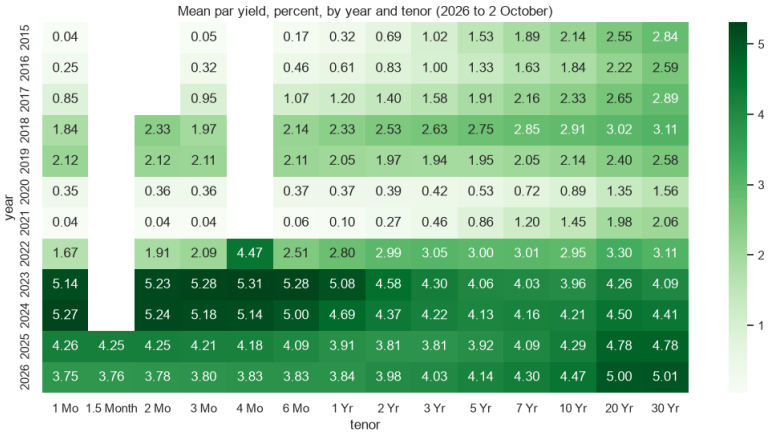
The 10 Yr year by year, and every tenor's mean by year

```
ten_year = yields.groupby(yields.index.year)['10 Yr'].agg(['mean', 'min', 'max']).round(2)
ten_year.index.name = 'year'
ten_year['range_bp'] = ((ten_year['max'] - ten_year['min']) * 100).round().astype(int)
ten_year
```

year	mean	min	max	range_bp
2015	2.14	1.68	2.50	82
2016	1.84	1.37	2.60	123
2017	2.33	2.05	2.62	57
2018	2.91	2.44	3.24	80
2019	2.14	1.47	2.79	132
2020	0.89	0.52	1.88	136
2021	1.45	0.93	1.74	81
2022	2.95	1.63	4.25	262
2023	3.96	3.30	4.98	168
2024	4.21	3.63	4.70	107
2025	4.29	3.97	4.79	82
2026	4.47	3.97	5.29	132

The mean was lowest in 2020, 0.89, and highest among whole years in 2025, 4.29; the range was widest in 2022, 262 bp, and narrowest in 2017, 57.

Averages by year say where each tenor sat. Next: when.



Each row is a year's average curve: pale in 2020 and 2021, dark in 2023 and 2024.

Excel side by side.

Job	pandas	Excel
A year's mean	`groupby(...).agg(...)`	=AVERAGEIFS(M:M, P:P, 2020) with =YEAR(A2) in P;
		MAXIFS, MINIFS from Excel 2019
Wide to long	`melt()`	Power Query, Unpivot Other Columns, which leaves out null cells
Shade a table	`sns.heatmap()`	Conditional Formatting, Color Scales

14.4, 14.5 Highs, lows, and three curves

```
for tenor in main:
    print(tenor, '| high', yields[tenor].max(), yields[tenor].idxmax().date(),
          '| low', yields[tenor].min(), yields[tenor].idxmin().date())
print('3 Mo at 0.00:', (yields['3 Mo'] == 0).sum(), 'dates')
3 Mo | high 5.63 2023-10-06 | low 0.0 2015-09-18
2 Yr | high 5.19 2023-10-17 | low 0.09 2021-02-05
10 Yr | high 5.29 2026-09-30 | low 0.52 2020-08-04
30 Yr | high 5.64 2026-09-30 | low 0.99 2020-03-09
3 Mo at 0.00: 13 dates
```

The "30 Yr" was highest on 2,335 dates and the "3 Mo" on 508, all 250 of 2023 among them.

The "10 Yr" peaked at 5.29 on 2026-09-30.

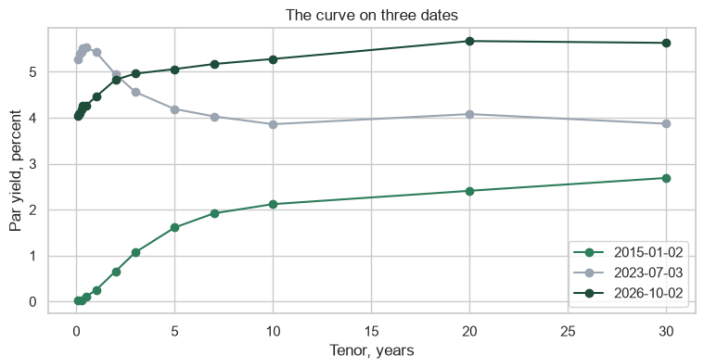
Excel side by side.

Job	pandas	Excel
Date of the high	`idxmax()`	=XLOOKUP(MAX(M2:M2941), M2:M2941, A2:A2941)
Month-end value	`resample('ME').last()`	=XLOOKUP(EOMONTH(DATE(2015,1,1),0), A2:A2941, M2:M2941, , -1), 1.68

```
furthest = (yields['10 Yr'] - yields['2 Yr']).idxmin()
dates = [yields.index[0], furthest, yields.index[-1]]
yields.loc[dates, ['1 Mo', '2 Yr', '10 Yr', '20 Yr', '30 Yr']]
```

date	1 Mo	2 Yr	10 Yr	20 Yr	30 Yr
2015-01-02	0.02	0.66	2.12	2.41	2.69
2023-07-03	5.27	4.94	3.86	4.08	3.87
2026-10-02	4.04	4.83	5.28	5.67	5.63

On 2023-07-03 the "2 Yr", 4.94, sat 108 bp above the "10 Yr", 3.86.



Excel side by side

Scatter with Straight Lines places each point by its x in years, as "ax.plot()" does; a Line chart spaces text labels evenly.

14.6 10 Yr less 2 Yr, and the days below zero

```
yields['spread_bp'] = ((yields['10 Yr'] - yields['2 Yr']) * 100).round()
print(yields['spread_bp'].iloc[0], yields['spread_bp'].iloc[-1])
print('mean', round(yields['spread_bp'].mean(), 1), '| median', yields['spread_bp'].median())
print('low', yields['spread_bp'].min(), yields['spread_bp'].idxmin().date())
print('high', yields['spread_bp'].max(), 'on',
      (yields['spread_bp'] == yields['spread_bp'].max()).sum(), 'dates')
146.0 45.0
mean 48.1 | median 50.0
low -108.0 2023-07-03
high 177.0 on 2 dates
```

```
below = yields['spread_bp'] < 0
print(below.sum(), 'days,', round(below.mean() * 100, 1), 'percent; at zero:',
      (yields['spread_bp'] == 0).sum())
print(yields.index[below][0].date(), 'to', yields.index[below][-1].date())
by_year_below = below.groupby(yields.index.year).sum()
by_year_below[by_year_below > 0]
544 days, 18.5 percent; at zero: 7
2019-08-27 to 2024-09-05
date
2019      3
2022    125
2023    250
2024    166
```



544 days below zero, every day of 2023. Last: how far yields move in a day.

It was below zero on 544 days, 18.5 percent of the file, from 2019-08-27 to 2024-09-05; "< 0" leaves out the 7 at exactly zero.

Excel side by side.

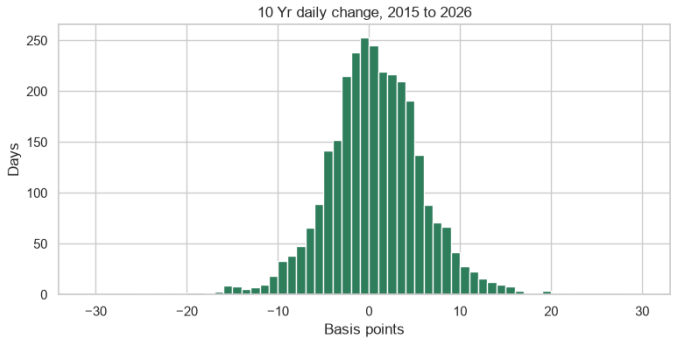
Job	pandas	Excel
Days below zero	`(yields['spread_bp'] < 0).sum()`	=COUNTIF(Q2:Q2941, "<0"), 544, with =(M2-I2)*100 filled down Q
In one year	`groupby(...).sum()`	=COUNTIFS(Q2:Q2941, "<0", P2:P2941, 2023), 250

14.7 How far yields move in a day

```
changes[main].describe().round(2)
```

	3 Mo	2 Yr	10 Yr	30 Yr
mean	0.14	0.14	0.11	0.10
std	2.95	5.19	5.30	5.05
min	-23.00	-57.00	-30.00	-31.00
25%	-1.00	-2.00	-3.00	-3.00
50%	0.00	0.00	0.00	0.00
75%	1.00	3.00	3.00	3.00
max	34.00	34.00	29.00	29.00

A typical "10 Yr" day is 5.30 bp, the middle half -3 to 3; the "3 Mo" moves least, 2.95.




```
print(changes['10 Yr'].nlargest(3))
print(changes['10 Yr'].nsmallest(3))
date
2020-03-17    29.0
2022-06-13    28.0
2020-03-10    22.0
Name: 10 Yr, dtype: float64
date
2022-11-10   -30.0
2022-09-28   -25.0
2023-03-10   -23.0
Name: 10 Yr, dtype: float64
```

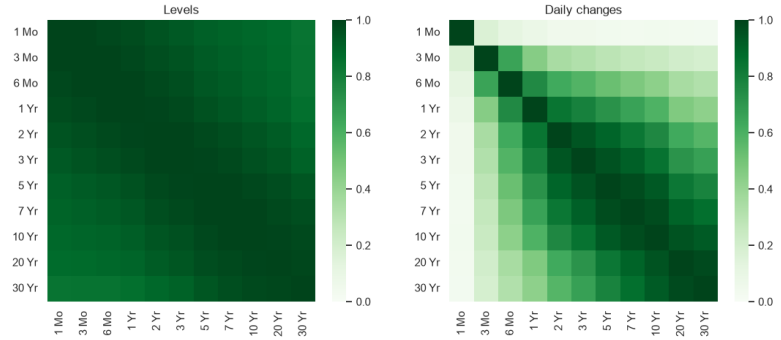
The largest rise, 29 bp on 2020-03-17; the largest fall, -30 on 2022-11-10.

The file says when, never why.

Moves of 10 bp or more came 55 times in 2023 and 48 in 2022, against 2 each in 2017 and 2018.

```
big = changes['10 Yr'].abs() >= 10
big.groupby(yields.index.year).sum().to_dict()
```

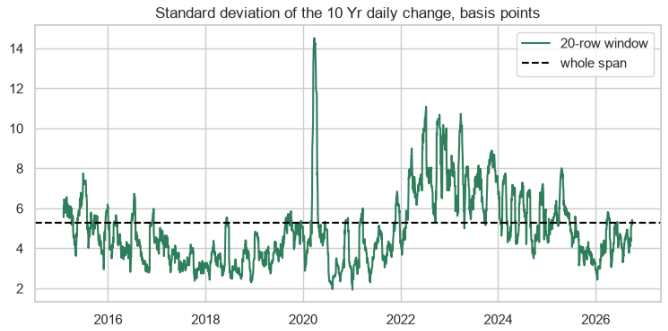
Tenors together, and the change over a 20-row window




```
for a, b in [('1 Mo', '30 Yr'), ('2 Yr', '10 Yr'), ('7 Yr', '10 Yr')]:
    print(a, b, round(yields[a].corr(yields[b]), 2), round(changes[a].corr(changes[b]), 2))
1 Mo 30 Yr 0.85 0.03
2 Yr 10 Yr 0.95 0.76
7 Yr 10 Yr 1.0 0.98
```

As levels, even the "1 Mo" and "30 Yr" correlate at 0.85: every tenor rose and fell with the years. As daily changes those two barely do, 0.03; neighbours still do, 0.98.

```
moving_std = changes['10 Yr'].rolling(20).std()
print('low', round(moving_std.min(), 2), moving_std.idxmin().date(),
      '| high', round(moving_std.max(), 2), moving_std.idxmax().date(),
      '| last', round(moving_std.iloc[-1], 2))
low 1.93 2021-01-05 | high 14.51 2020-03-26 | last 5.34
```



It ran from 1.93 to 14.51 bp and reads 5.34 on the last date, against 5.30 for the span: a description, not a forecast.

Excel side by side.

Job	pandas	Excel
Daily change	`diff() * 100`	=(M3-M2)*100 in R3, filled down
Largest moves	`nlargest(), nsmallest()`	=LARGE(R3:R2941, 1), =SMALL(R3:R2941, 1)
20-row average	`rolling(20).mean()`	=AVERAGE(M2:M21) filled down, or a Moving Average trendline, period 20

5.30 bp a day over the span, 5.34 on the last date: the sheet's last line.

Predict first: decide, then run each in the notebook

Predict 1

2020 had 366 days. What shape does "yields.loc['2020']" have? A) (1, 14) B) (366, 14) C) about 250 rows, 14 columns

```
print(yields.loc['2020'].shape)
print(yields.loc['2020-03'].shape)
```

Predict 2

2019-12-30 to 2020-01-03 is five weekdays, and a label slice keeps its end, as in pandas Part 1. How many rows come back? A) 5 B) 4 C) 3

```
yields.loc['2019-12-30':'2020-01-03', ['2 Yr', '10 Yr']]
```

Predict 3

"yields" has 2,940 rows, 14 tenors, and 5,432 blank cells. What shape does the long table have? A) (2940, 3) B) (35728, 3) C) (41160, 3)

```
long = yields.reset_index().melt(id_vars='date', var_name='tenor', value_name='yield_pct')
print(long.shape)
long.head(3)
```

Predict first: month ends, changes, and a window

Predict 4

"resample('ME')" groups the rows by calendar month; ".last()" keeps each month's last value. January 2015 to October 2026: what shape comes back? A) (12, 14) B) (141, 14) C) (142, 14)

```
month_end = yields[tenors].resample('ME').last()
print(month_end.shape)
print(month_end[['2 Yr', '10 Yr']].tail(2))
print('2015-01-31' in yields.index, month_end['10 Yr'].iloc[0],
      round(yields.loc['2015-01', '10 Yr'].mean(), 2))
```

Predict 5

"diff()" takes each row less the row before. The file has 2,940 rows. How many "10 Yr" changes come back? A) 2940 B) 2939 C) 2938

```
changes = (yields[tenors].diff() * 100).round()
print(changes['10 Yr'].count())
changes['10 Yr'].head(3)
```

Predict 6

"rolling(20).mean()" averages the 20 rows ending on each date. How many "NaN" does "yields['10 Yr'].rolling(20).mean()" leave? Write a number.

```
average_20 = yields['10 Yr'].rolling(20).mean()
print('missing:', average_20.isna().sum())
print(round(average_20.dropna().iloc[0], 2), average_20.first_valid_index().date())
```

The note, answered

The note asked	The file says
Covered, missing	2015-01-02 to 2026-10-02; 126 weekdays absent, 5,432 cells blank
Late tenors	2 Mo from 2018-10-16, 4 Mo from 2022-10-19, 1.5 Month from 2025-02-18
By year	10 Yr mean 0.89 (2020) to 4.29 (2025); widest range 262 bp (2022)
Highs and lows	10 Yr 5.29 on 2026-09-30, 0.52 on 2020-08-04
The curve	2023-07-03: the 2 Yr 108 bp above the 10 Yr
10s less 2s	below zero 544 days, all 250 of 2023
A day's move	5.30 bp over the span, 14.51 at the widest window, 5.34 on the last date

Be proud: the course's first real file, and every sentence a number it supports.

Today's file was the market's; Day 5 opens the desk's own trade blotter: what the desk did.